

Usuarios

José Calderón, Fernando Sola, Daniel Ayala, Inma Hernández, Margarita Cruz, Carlos Arévalo y David Ruiz



Escuela Técnica Superior de
Ingeniería Informática

1. Requisitos de información (RI)	2
1.1. RI-1: Usuarios	2
2. Reglas de negocio (RN)	3
2.1. RN-1: Mayor de edad	3
2.2. RN-2: Unicidad del correo electrónico	3
3. Requisitos funcionales (RF)	4
3.1. RF-1: Informes simples de Usuarios	4
3.2. RF-2: Informes complejos de Usuarios	4
4. Pruebas de aceptación (PA)	6
4.1. PA-1: Usuarios	6
5. Diagrama de clases	7
6. Álgebra relacional	9
7. Script SQL para crear la base de datos	13
8. Script SQL para la carga inicial de datos	15
9. Consultas	17
10. SQL avanzado	19
11. Fecha de nacimiento en lugar de la edad	21
12. Pruebas SQL	25

Requisitos

Requisitos de información (RI)

1.1. RI-1: Usuarios

- Como: Profesor de la asignatura
- Quiero: Poder almacenar la siguiente información de los usuarios: nombre, edad, email y género (masculino, femenino, otro). Todos los atributos son obligatorios salvo el género.
- Para: Que el estudiante realice a partir de este requisito el modelo conceptual, relacional y tecnológico.

Reglas de negocio (RN)

2.1. RN-1: Mayor de edad

- Como: Profesor de la asignatura
- Quiero: Que los usuarios del sistema sean mayores de edad
- Para: Que el estudiante practique con restricciones simples.

2.2. RN-2: Unicidad del correo electrónico

- Como: Profesor de la asignatura
- Quiero: Que no existan dos usuarios en el sistema con el mismo correo electrónico
- Para: Que el estudiante practique con restricciones simples.

Requisitos funcionales (RF)

3.1. RF-1: Informes simples de Usuarios

- Como: Profesor de la asignatura
- Quiero: Que el sistema sea capaz de generar los siguientes informes:
 - Usuarios ordenados por nombre de la A a la Z
 - Nombre y correo de los usuarios de género femenino.
 - Nombre, edad y correo de los usuarios con dominio '@us.es'.
 - Edad media y total de usuarios.
 - Edad media y total de los usuarios con dominio '@us.es'.
- Para: Que el estudiante practique con consultas simples.

3.2. RF-2: Informes complejos de Usuarios

- Como: Profesor de la asignatura
- Quiero: Que el sistema sea capaz de generar los siguientes informes:
 - Edad media de los usuarios según el género.
 - Número de usuarios según el género.
 - Edad media de los usuarios según el género.
 - Total de usuarios según el género.
 - Usuarios de mayor edad.
 - Usuarios de mayor edad según el género.

3. Requisitos funcionales (RF)

- Edad media y total de usuarios según el dominio de su correo electrónico.
- Para: Que el estudiante practique con consultas complejas.

Pruebas de aceptación (PA)

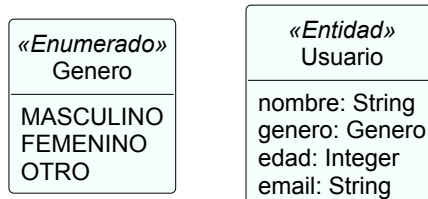
4.1. PA-1: Usuarios

1. ☒ Crear un nuevo usuario con todos los datos correctos según las reglas de negocio
2. ☒ Crear un nuevo usuario sin especificar el género.
3. ☐ Crear un nuevo usuario sin nombre.
4. ☐ Crear un nuevo usuario sin edad.
5. ☐ Crear un nuevo usuario sin email.
6. ☐ Crear un nuevo usuario con el correo repetido.
7. ☐ Crear un nuevo usuario menor de edad.

Modelo Conceptual

Diagrama de clases

Usuarios: caso base



RN01: Usuarios mayores de edad
RN02: Email único por usuario

Modelo Relacional

```
-- Intensión
Usuarios = { usuarioId, nombre, edad, género, email }
PK(usuarioId)
AK(email)

-- Extensión
Usuarios = {
    (u1, "David Ruiz", 45, MASCULINO, "druiz@us.es"),
    (u2, "Carlos Arévalo", 58, MASCULINO, "carevalo@us.es"),
    (u3, "Margarita Cruz", 58, FEMENINO, "mcruz@us.es"),
    (u4, "Inma Hernández", 35, FEMENINO, "inmahernandez@us.es"),
    (u5, "Alfonso Márquez", 35, MASCULINO, "amarquez@us.es"),
    (u6, "Daniel Ayala", 28, MASCULINO, "dayalal@us.es"),
    (u7, "Raquel Sampedro", 55, FEMENINO, "rsampedro@gmail.com"),
}
```

```
(u8, "Marta López", 18, FEMENINO, "mlopez@mail.com"),  
(u9, "David Ruiz", 25, MASCULINO, "druiz@mail.com"),  
(u10, "Andrea Gómez", 27, OTRO, "agomez@mail.es"),  
(u11, "Ernesto Murillo", 55, OTRO, "emurillo@correo.es")  
}
```

Álgebra relacional

- Renombrado de la relación Usuarios (corregido para incluir el atributo género):

$$\rho_{U(uid,n,ed,g,em)}(Usuarios)$$

- Nombre y correo de las usuarias (género femenino):

$$Mujeres \leftarrow \Pi_{n,em} \left(\sigma_{g=FEMENINO}(U) \right)$$

```
Mujeres = { n, em }
```

```
Mujeres = {
  ("Margarita Cruz", "mcruz@us.es"),
  ("Inma Hernández", "inmahernandez@us.es"),
  ("Raquel Sampedro", "rsampedro@gmail.com"),
  ("Marta López", "mlopez@mail.com")
}
```

- Nombre, edad y correo de los usuarios con dominio "@us.es":

$$UsuariosUS \leftarrow \Pi_{n,ed,em} \left(\sigma_{dominio(em)='us.es'}(U) \right)$$

```
UsuariosUS = { n, ed, em }
```

```
UsuariosUS = {
  ("David Ruiz", 45, "druiz@us.es"),
  ("Carlos Arévalo", 58, "carevalo@us.es"),
  ("Margarita Cruz", 58, "mcruz@us.es"),
  ("Inma Hernández", 35, "inmahernandez@us.es"),
  ("Alfonso Márquez", 35, "amarquez@us.es"),
}
```

```
("Daniel Ayala", 28, "dayalal@us.es")
}
```

Asumimos una función `dominio()` que dado un email devuelve su dominio (la cadena tras la @).

- Edad media y total de usuarios:

$$MedTotUsuarios \leftarrow \rho_{media}(AVG(ed)), \rho_{total}(COUNT(*))_{\gamma}(U)$$

```
MedTotUsuarios = { media, total }

MedTotUsuarios = {
  (39.91, 11)
}
```

- Edad media y total de los usuarios con dominio "@us.es":

$$MedTotUsuariosUS \leftarrow \rho_{media}(AVG(ed)), \rho_{total}(COUNT(*))_{\gamma}(UsuariosUS)$$

```
MedTotUsuariosUS = { media, total }

MedTotUsuariosUS = {
  (43.17, 6)
}
```

- Edad media de los usuarios agrupada por género:

$$MediaGenero \leftarrow g, \rho_{media}(AVG(ed))_{\gamma} g(U)$$

```
MediaGenero = { g, media }

MediaGenero = {
  (MASCULINO, 38.20),
  (FEMENINO, 41.50),
  (OTRO, 41.00)
}
```

- Número de usuarios agrupados por género:

$$TotalGenero \leftarrow g, \rho_{total}(COUNT(*))_{\gamma} g(U)$$

```
TotalGenero = { g, total }

TotalGenero = {
  (MASCULINO, 5),
  (FEMENINO, 4),
  (OTRO, 2)
}
```

- Edad media y total de usuarios según el dominio del correo electrónico:

$$MedTotDominio \leftarrow \gamma_{\text{dominio}(em), \rho_{media}(\text{AVG}(ed)), \rho_{total}(\text{COUNT}(*))} \text{dominio}(em)(U)$$

```
MedTotDominio = { dominio, media, total }
```

```
MedTotDominio = {
  ("us.es", 43.17, 6),
  ("gmail.com", 55.00, 1),
  ("mail.com", 21.50, 2),
  ("mail.es", 27.00, 1),
  ("correo.es", 55.00, 1)
}
```

- Edad del usuario de mayor edad:

$$edadMayor \leftarrow \gamma_{\rho_{mayor}(\text{MAX}(ed))} (U)$$

```
edadMayor = { mayor }
```

```
edadMayor = {
  (58)
}
```

- Usuarios de mayor edad:

$$UsuariosMayores \leftarrow \Pi_{uid, n, ed, g, em} \left(\sigma_{ed=mayor} (U \times edadMayor) \right)$$

```
UsuariosMayores = { uid, n, ed, g, em }
```

```
UsuariosMayores = {
  (u2, "Carlos Arévalo", 58, MASCULINO, "carevalo@us.es"),
  (u3, "Margarita Cruz", 58, FEMENINO, "mcruz@us.es")
}
```

- Edad máxima por género:

$$MayoresGenero \leftarrow \gamma_{g, \rho_{mayor}(\text{MAX}(ed))} g(U)$$

```
MayoresGenero = { g, mayor }
```

```
MayoresGenero = {
  (MASCULINO, 58),
  (FEMENINO, 58),
  (OTRO, 55)
}
```

- Usuarios de mayor edad según el género:

$$UsuariosMayoresGenero \leftarrow \Pi_{uid, n, ed, g, em} \left(\sigma_{ed=mayor} (U \bowtie MayoresGenero) \right)$$

```
UsuariosMayoresGenero = { uid, n, ed, g, em }
```

```
UsuariosMayoresGenero = {  
  (u2, "Carlos Arévalo", 58, MASCULINO, "carevalo@us.es"),  
  (u3, "Margarita Cruz", 58, FEMENINO, "mcruz@us.es"),  
  (u11, "Ernesto Murillo", 55, OTRO, "emurillo@correo.es")  
}
```

Modelo Tecnológico Para crear el esquema de la base de datos en MariaDB se puede usar el siguiente script:

Script SQL para crear la base de datos

usuarios/createDB.sql

```
--
-- Autor: David Ruiz
-- Fecha: Noviembre 2024
-- Descripción: Script para crear la BD del ejercicio de Usuarios
--

DROP DATABASE IF EXISTS UsersDB;
CREATE DATABASE UsersDB;
USE UsersDB;

--
=====
-- Eliminamos tablas previas
--
=====

SET FOREIGN_KEY_CHECKS = 0;
DROP TABLE IF EXISTS users;
SET FOREIGN_KEY_CHECKS = 1;

--
=====
-- Tabla: users
--
```

```
=====
CREATE TABLE users (
  user_id INT AUTO_INCREMENT,
  full_name VARCHAR(120) NOT NULL,
  gender ENUM('MASCULINO', 'FEMENINO', 'OTRO') NOT NULL,
  age TINYINT NOT NULL,
  email VARCHAR(255) NOT NULL,
  PRIMARY KEY (user_id),
  CONSTRAINT rn01_users_adult_age CHECK (age >= 18),
  CONSTRAINT rn02_users_unique_email UNIQUE (email)
);
```

Script SQL para la carga inicial de datos

Para cargar los datos de prueba se puede usar el siguiente script:

usuarios/populateDB.sql

```
--
-- Autor: David Ruiz
-- Fecha: Noviembre 2024
-- Descripción: Procedimiento para poblar la BD de Usuarios
--
USE UsersDB;

DELIMITER //
CREATE OR REPLACE PROCEDURE p_populate()
BEGIN
    DELETE FROM users;
    ALTER TABLE users AUTO_INCREMENT = 1;

    INSERT INTO users (full_name, gender, age, email) VALUES
        ('David Ruiz', 'MASCULINO', 45, 'druiz@us.es'),
        ('Carlos Arevalo', 'MASCULINO', 58, 'carevalo@us.es'),
        ('Margarita Cruz', 'FEMENINO', 58, 'mcruz@us.es'),
        ('Inma Hernandez', 'FEMENINO', 35, 'inmahernandez@us.es'),
        ('Alfonso Marquez', 'MASCULINO', 35, 'amarquez@us.es'),
        ('Daniel Ayala', 'MASCULINO', 28, 'dayalal@us.es'),
        ('Raquel Sampedro', 'FEMENINO', 55, 'rsampedro@gmail.com'),
        ('Marta Lopez', 'FEMENINO', 18, 'mlopez@mail.com'),
```

```
      ('David Ruiz', 'MASCULINO', 25, 'druiz@mail.com'),  
      ('Andrea Gomez', 'OTRO', 42, 'agomez@mail.es'),  
      ('Ernesto Murillo', 'OTRO', 55, 'emurillo@correo.es');  
END //  
DELIMITER ;  
  
CALL p_populate();
```

Consultas

Para crear las consultas que implementan los requisitos funcionales se puede usar el siguiente script:

usuarios/queries.sql

```
--  
-- Autor: David Ruiz  
-- Fecha: Noviembre 2024  
-- Descripción: Consultas de referencia para la BD de Usuarios  
--  
  
USE UsersDB;  
  
-- RF01: Usuarios ordenados alfabéticamente  
SELECT *  
FROM users  
ORDER BY full_name ASC;  
  
-- RF02: Nombre y correo de las usuarias  
SELECT full_name, email  
FROM users  
WHERE gender = 'FEMENINO';  
  
-- RF03: Usuarios con dominio corporativo us.es  
SELECT full_name, age, email  
FROM users  
WHERE email LIKE '%@us.es';
```

```

-- RF04: Estadísticas globales de edad
SELECT AVG(age) AS average_age, COUNT(*) AS total_users
FROM users;

-- RF05: Estadísticas de edad y total por dominio
SELECT f_get_email_domain(email) AS email_domain,
       AVG(age) AS average_age,
       COUNT(*) AS total_users
FROM users
GROUP BY email_domain
ORDER BY total_users DESC;

-- RF06: Distribución por género
SELECT gender, AVG(age) AS average_age, COUNT(*) AS total_users
FROM users
GROUP BY gender;

-- RF07: Usuarios de mayor edad
SELECT *
FROM users
WHERE age = (SELECT MAX(age) FROM users);

-- RF08: Usuario de mayor edad por género
SELECT u1.*
FROM users u1
WHERE age = (
    SELECT MAX(u2.age)
    FROM users u2
    WHERE u2.gender = u1.gender
);

```

SQL avanzado

Tenemos que implementar una función que calcula la edad de un usuario a partir de su fecha de nacimiento, y un trigger que comprueba que la fecha de nacimiento es anterior a la fecha actual y, además el usuario es mayor de edad.

El script para la función es el siguiente:

usuarios/fGetAge.sql

```
--  
-- Autor: David Ruiz  
-- Fecha: Noviembre 2024  
-- Descripción: Función auxiliar para calcular la edad a partir de una  
-- fecha  
--  
  
USE UsersDB;  
  
DELIMITER //  
CREATE OR REPLACE FUNCTION f_get_age(p_birth_date DATE)  
RETURNS TINYINT  
DETERMINISTIC  
BEGIN  
    RETURN TIMESTAMPDIF(YEAR, p_birth_date, CURDATE());  
END //  
DELIMITER ;
```

El script para obtener el dominio de un correo electrónico es el siguiente:

usuarios/fEmailDomain.sql

```
--
-- Autor: David Ruiz
-- Fecha: Noviembre 2024
-- Descripción: Función para obtener el dominio de un email
--

USE UsersDB;

-- Cálculo del dominio en SQL: https://stackoverflow.com/questions/2628138/
-- how-to-select-domain-name-from-email-address
DELIMITER //
CREATE OR REPLACE FUNCTION f_get_email_domain(p_email VARCHAR(255))
RETURNS VARCHAR(255)
DETERMINISTIC
BEGIN
    RETURN SUBSTRING_INDEX(SUBSTR(p_email, INSTR(p_email, '@') + 1), ' ',
1);
END //
DELIMITER ;
```

El script para el trigger es el siguiente:

usuarios/tCheckAge.sql

```
--
-- Autor: David Ruiz
-- Fecha: Noviembre 2024
-- Descripción: Trigger para validar la edad mínima de los usuarios
--

USE UsersDB;

DELIMITER //
CREATE OR REPLACE TRIGGER t_bi_users_rn01
BEFORE INSERT ON users
FOR EACH ROW
BEGIN
    IF NEW.age < 18 THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'RN01: Los usuarios deben ser mayores de
edad';
    END IF;
END //
DELIMITER ;
```

Fecha de nacimiento en lugar de la edad

En la creación de la tabla ahora hay que añadir que sustituir la edad por la fecha de nacimiento, pero ya no se puede comprobar que el Usuario es mayor de edad con un simple CHECK, y es necesario usar un TRIGGER, además para calcular la edad definiremos una función.

usuarios/version2.sql

```
--
-- Autor: David Ruiz
-- Fecha: Noviembre 2024
-- Descripción: Variante didáctica de Usuarios donde se guarda la fecha de
-- nacimiento
--              y la edad se calcula con una función. No se usa en el
proyecto real,
--              solo como ejemplo para alumnos.
--

USE UsersDB;

-- =====
-- Ejemplo de tabla con fecha de nacimiento en lugar de campo age entero
-- =====
-- DROP TABLE IF EXISTS users;
-- CREATE TABLE users (
--     user_id INT AUTO_INCREMENT,
--     full_name VARCHAR(120) NOT NULL,
--     gender ENUM('MASCULINO', 'FEMENINO', 'OTRO') NOT NULL,
```

```

--      birth_date DATE NOT NULL,
--      email VARCHAR(255) NOT NULL,
--      PRIMARY KEY (user_id),
--      CONSTRAINT rn01_users_adult_age CHECK (TIMESTAMPDIFF(YEAR,
birth_date, CURDATE()) >= 18),
--      CONSTRAINT rn02_users_unique_email UNIQUE (email)
-- );

-- =====
-- Populate alternativo usando birth_date
-- =====
-- DELIMITER //
-- CREATE OR REPLACE PROCEDURE p_populate_users_v2()
-- BEGIN
--     DELETE FROM users;
--     ALTER TABLE users AUTO_INCREMENT = 1;

--     INSERT INTO users (full_name, gender, birth_date, email) VALUES
--         ('David Ruiz', 'MASCULINO', '1979-05-18', 'druiz@us.es'),
--         ('Carlos Arévalo', 'MASCULINO', '1966-06-12', 'carevalo@us.es'),
--         ('Margarita Cruz', 'FEMENINO', '1966-12-01', 'mcruz@us.es'),
--         ('Inma Hernández', 'FEMENINO', '1989-03-11',
'inmahernandez@us.es'),
--         ('Alfonso Márquez', 'MASCULINO', '1989-04-12',
'amarquez@us.es'),
--         ('Daniel Ayala', 'MASCULINO', '1996-05-13', 'dayalal@us.es'),
--         ('Raquel Sampedro', 'FEMENINO', '1969-09-07',
'rsampedro@gmail.com'),
--         ('Marta López', 'FEMENINO', '2006-09-07', 'mlopez@mail.com'),
--         ('David Ruiz', 'MASCULINO', '1999-09-07', 'druiz@mail.com'),
--         ('Andrea Gómez', 'OTRO', '1982-01-10', 'agomez@mail.es'),
--         ('Ernesto Murillo', 'OTRO', '1969-02-15', 'emurillo@correo.es');
-- END //
-- DELIMITER ;

-- =====
-- Función para calcular la edad a partir de la fecha de nacimiento
-- =====
DELIMITER //
CREATE OR REPLACE FUNCTION f_get_age_from_birth_date(p_birth_date DATE)
RETURNS TINYINT
DETERMINISTIC
BEGIN
    RETURN TIMESTAMPDIFF(YEAR, p_birth_date, CURDATE());
END //
DELIMITER ;

-- =====
-- Ejemplos de uso
-- =====

```

11. Fecha de nacimiento en lugar de la edad

```
-- SELECT full_name, birth_date, f_get_age_from_birth_date(birth_date) AS
age
-- FROM users;
--
-- INSERT INTO users (full_name, gender, birth_date, email)
-- VALUES ('Alumno Ejemplo', 'MASCULINO', '2000-05-10', 'ejemplo@us.es');

-- =====
-- Consultas adaptadas a birth_date (en lugar de age)
-- =====
-- RF01: Usuarios ordenados alfabéticamente
-- SELECT * FROM users ORDER BY full_name ASC;

-- RF02: Nombre y correo de las usuarias
-- SELECT full_name, email FROM users WHERE gender = 'FEMENINO';

-- RF03: Usuarios con dominio us.es
-- SELECT full_name, f_get_age_from_birth_date(birth_date) AS age, email
-- FROM users
-- WHERE email LIKE '%@us.es';

-- RF04: Estadísticas globales de edad
-- SELECT AVG(f_get_age_from_birth_date(birth_date)) AS average_age,
COUNT(*) AS total_users
-- FROM users;

-- RF05: Estadísticas de edad y total por dominio
-- SELECT f_get_email_domain(email) AS email_domain,
--        AVG(f_get_age_from_birth_date(birth_date)) AS average_age,
--        COUNT(*) AS total_users
-- FROM users
-- GROUP BY email_domain
-- ORDER BY total_users DESC;

-- RF06: Distribución por género
-- SELECT gender,
--        AVG(f_get_age_from_birth_date(birth_date)) AS average_age,
--        COUNT(*) AS total_users
-- FROM users
-- GROUP BY gender;

-- RF07: Usuarios de mayor edad
-- SELECT * FROM users
-- WHERE f_get_age_from_birth_date(birth_date) = (
--        SELECT MAX(f_get_age_from_birth_date(birth_date)) FROM users
-- );

-- RF08: Usuario de mayor edad por género
-- SELECT u1.*
-- FROM users u1
```

```
-- WHERE f_get_age_from_birth_date(birth_date) = (  
--     SELECT MAX(f_get_age_from_birth_date(birth_date))  
--     FROM users u2  
--     WHERE u2.gender = u1.gender  
-- );
```

Pruebas SQL

usuarios/tests.sql

```
--
-- Autor: David Ruiz
-- Fecha: Noviembre 2024
-- Descripción: Tests negativos para la BD de Users
--
USE UsersDB;

-- =====
-- TABLA DE RESULTADOS
-- =====
CREATE OR REPLACE TABLE test_results (
    test_id VARCHAR(20) PRIMARY KEY,
    test_name VARCHAR(200) NOT NULL,
    test_message VARCHAR(500) NOT NULL,
    test_status ENUM('PASS', 'FAIL', 'ERROR') NOT NULL,
    execution_time TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- =====
-- PROCEDIMIENTO AUXILIAR
-- =====
DELIMITER //
CREATE OR REPLACE PROCEDURE p_log_test(
    IN p_test_id VARCHAR(20),
    IN p_message VARCHAR(500),
    IN p_status ENUM('PASS', 'FAIL', 'ERROR')
```

```

)
BEGIN
    INSERT INTO test_results(test_id, test_name, test_message, test_status)
    VALUES (p_test_id, SUBSTRING_INDEX(p_message, ':', 1), p_message,
p_status);
END //

-- =====
-- TESTS (RN01 - RN02)
-- =====

CREATE OR REPLACE PROCEDURE p_test_rn01_minimum_age()
BEGIN
    DECLARE EXIT HANDLER FOR SQLEXCEPTION
        CALL p_log_test('RN01', 'RN01: Los usuarios deben ser mayores de
edad', 'PASS');

    CALL p_populate();

    INSERT INTO users (full_name, gender, age, email)
    VALUES ('Usuario Menor', 'MASCULINO', 17, 'menor@example.com');

    CALL p_log_test('RN01', 'ERROR: Se permitió registrar un usuario menor
de edad', 'FAIL');
END //

CREATE OR REPLACE PROCEDURE p_test_rn02_unique_email()
BEGIN
    DECLARE EXIT HANDLER FOR SQLEXCEPTION
        CALL p_log_test('RN02', 'RN02: No se permiten emails duplicados',
'PASS');

    CALL p_populate();

    INSERT INTO users (full_name, gender, age, email)
    VALUES ('Correo Duplicado', 'FEMENINO', 30, 'druiz@us.es');

    CALL p_log_test('RN02', 'ERROR: Se permitió duplicar un email',
'FAIL');
END //

-- =====
-- ORQUESTADOR
-- =====

CREATE OR REPLACE PROCEDURE p_run_tests()
BEGIN
    DECLARE EXIT HANDLER FOR SQLEXCEPTION
    BEGIN
        CALL p_log_test('POPULATE', 'ERROR: El populate falló. No se
ejecutaron los tests negativos.', 'ERROR');
    END

```

```
SELECT * FROM test_results ORDER BY execution_time, test_id;
SELECT test_status, COUNT(*) AS total FROM test_results GROUP BY
test_status;
END;

DELETE FROM test_results;

CALL p_populate();

CALL p_test_rn01_minimum_age();
CALL p_test_rn02_unique_email();

SELECT * FROM test_results ORDER BY execution_time, test_id;
SELECT test_status, COUNT(*) AS total FROM test_results GROUP BY
test_status;
END //
DELIMITER ;

CALL p_run_tests();
```

i Info

Versión PDF disponible